

Luiz
Pedro Augusto dos Santos Kzan 12547465

SEL0337 Projetos em Sistemas Embarcados

Prof. Dr. Pedro de Oliveira Conceição Júnior

Relatório 4

Parte 1.1: LED por I2C.

Códigos utilizados:

Código utilizado no Raspberry Pi:

```
# Tutorial: Raspberry Pi controlando o Arduino

# sudo raspi-config - interface options - habilitar I2C

# sudo i2cdetect -y 1 - verifica o barramento I2C - ao realizar a conexão I2C física com Arduino o
# endereço ocupado deve aparecer

from smbus import SMBus # importa a classe SMBus

addr = 0x8 # bus address - define o endereço do dispositivo I2C como 0x8 (hexa)

bus = SMBus(1) # /dev/ic2-1 - inicializa o objeto, criando uma instância para se comunicar com
# barramento I2C nº 1

flag = True # variv. usada p/ controlar o loop

print ("Digite 1 para ON ou 0 para OFF")

while flag:

    ledstate = input(">>>> ") # solicita entrada no terminal

    if ledstate == "1":

        bus.write_byte(addr, 0x1) # se for 1, escreve 0x1 no endereço
```

```

elif ledstate == "0":

bus.write_byte(addr, 0x0) # ou escreve 0x0, caso 0

else:

flag = False # encerra o loop caso digite algo diferente de 0 ou 1

# Os valores de entrada acima, enviam 1 ou 0 para o barramento I2C - ou 0 em caso de valor de
entrada diferente de 0 ou 1.

# Até aqui, o código acima, se testado, deve ser capaz de controlar o LED do Arduino, apagando
(quando enviado 0) ou ligando (quando enviado 1). Para que o Arduino recebe o controle, deve ser
feito o upload de um programa em C, via IDE. O tutorial encontra-se abaixo.

# A partir daqui, conforme solicitado no roteiro da prática, a linha abaixo deve ser configurada para a
leitura de dois bytes - referente aos valores analógicos que serão lidos do potenciômetro conectado ao
Arduino - na resolução de 10 bits

# <Bus.read_i2c_block_data(endereço do arduino, registrador, conjunto debytes)>

# exemplo: data = bus.read_i2c_block_data(0x8, 0, 2)

# por fim, realizar a conversão da saída acima, de 0 e 1023 (10 bits) recebidos do Arduino, para
imprimir no terminal da Rasp o valor entre 0 e 255 (8 bits)

#exemplo: value = data[0]*256+data[1]

```

Código utilizado no Arduino:

```

// código em C a ser compilado na IDE do Arduino e feito upload na placa

// biblioteca Wire para I2C

#include <Wire.h>

// Controlar o LED da própria placa Arduino - LED_BUILTIN é uma constante que se refere ao pino
do LED embutido na placa

const int ledPin = LED_BUILTIN;

void setup() {

// adicionando endereço no barramento I2C com dispositivo controlado e inicia a comunicação

Wire.begin(0x8);

```

```

//Reportar "receiveEvent" quando receber dados

Wire.onReceive(receiveEvent);

// Define o pino do LED como saída e o deixa desligado inicialmente

pinMode(ledPin, OUTPUT);

digitalWrite(ledPin, LOW);

}

// Definição da Função receiveEvent executada sempre que dados são recebidos do controlador
(Raspberry Pi)

void receiveEvent(int howMany) {

while (Wire.available()) { // loop enquanto houver dados no barramento

char c = Wire.read(); // recebe o byte como char

digitalWrite(ledPin, c); // define o estado do LED com base no valor de "c"

}

}

void loop() { // aguarda 100 ms antes de continuar a execução em loop delay(100);

}

// até aqui, o código acima, se testado, deve atender às solicitações da Rasp, ligando ou desligando o
LED do Arduino.

// A partir daqui, o script deve ser configurado visando a leitura analógica do potenciômetro, que será
solicitada na Rasp

// void setup () {

// o parâmetro <Serial.begin(9600)> define a taxa de comunicação serial - 9600 bits por segundo,
no caso

//a função < Wire.onRequest(requestEvents)> é chamada quando o Arduino receber solicitações da
Rasp

// }

// void requestEvent () {

// a função: <analogRead(analog_pin)> realiza leitura do pino analogico que se deseja receber dados

```

```
// <Wire.write(highByte(n)); > e < Wire.write(lowByte(n));> são funções usadas na chamada da  
função de envio de bytes pelo I2C por highByte e lowByte
```

```
// }
```

```
// void receiveEvent(int number) {
```

```
// sem alterações
```

```
//}
```

```
// Lembrar sempre de, ao longo dos blocos, imprimir na tela os valores com < Serial.println>
```

Os códigos foram copiados do roteiro da prática sem alterações, e funcionaram perfeitamente, como é mostrado no seguinte vídeo: <<https://youtu.be/SaGSYWkmdqA>>

Parte 1.2.: Leitura de potenciômetro por I2C.

Montagem:

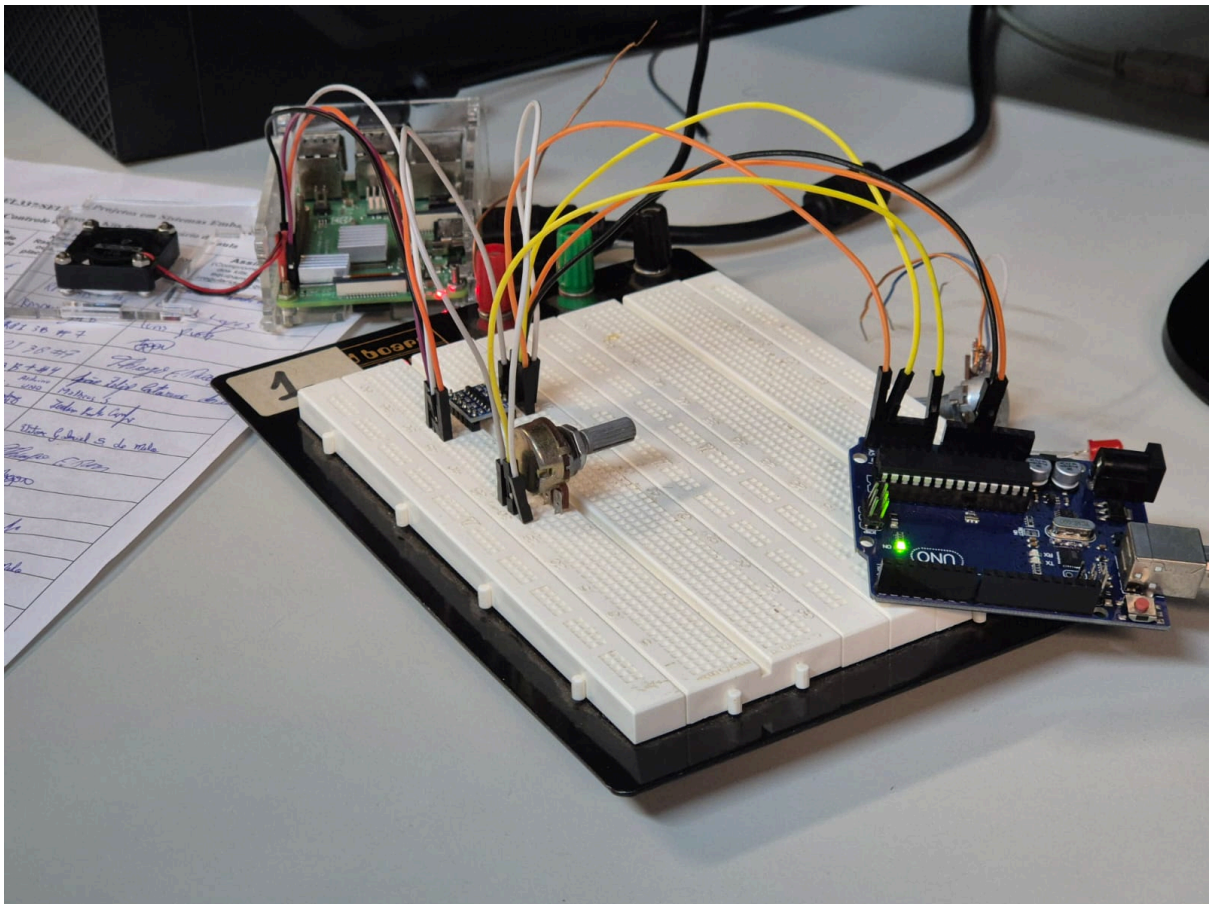


Figura: montagem de acordo com o roteiro para a leitura por i2c da posição do potenciômetro usando Arduino e Raspberry Pi.

Códigos utilizados:

Código utilizado no Arduino:

```
//  
  
//  
  
//  
  
#include <Wire.h>  
  
  
const int LedPin = LED_BUILTIN;  
  
  
int x;  
  
  
void setup() {  
  
    pinMode(LedPin, OUTPUT);  
  
    pinMode(A0, INPUT);  
  
    digitalWrite(LedPin, LOW);  
  
  
    Wire.begin(0x08);  
  
    Wire.onReceive(Detection);  
  
  
    Serial.begin(9600);  
  
    Wire.onRequest(requestEvent);
```

```

// put your setup code here, to run once:

}

void Detection(int count_on){
while(Wire.available()){
    char a = Wire.read();
    digitalWrite(LedPin, a);
}

}

void loop() {
    delay(100);

    // put your main code here, to run repeatedly:

}

```

```

void requestEvent (int x) {
x = analogRead(A0);
Serial.println(x);
Wire.write(highByte(x));
Wire.write(lowByte(x));
}

```

Possui leitura analógica em A0 para ler a posição do potenciômetro, e encaminha em dois Bytes a informação no endereço 0x08.

Código utilizado no Raspberry Pi:

```
from smbus import SMBus # importa a classe SMBus

from time import sleep

addr = 0x8

bus = SMBus(1)

flag = True


while flag:

    data = bus.read_i2c_block_data(0x8, 0, 2) # define o
endereço de memoria

    value = data[0]*256+data[1] # define o valor registrado no
endereço

    print (value)


    sleep(1)
```

O funcionamento desta parte da prática deu certo, e o vídeo mostrando ele foi o seguinte:
<<https://youtu.be/no5QtIT92TU>>.

Referências:

Roteiro da prática, Professor, Monitores